```bash
dir
```

and send me the output.

Important

Do not modify the code yet.

We need the original game running first because the assignment explicitly requires a **10-second recording before making changes**. Lab_4_VibeCoding_Student_handout

If you get an error, **paste the complete terminal error here**, rather than trying five random Stack Overflow rituals. We'll fix the original setup first.

ok the main.py works and i did a 20 sec recorfing, next

Worked for 8s

We're ready for Task 1

The README gives us the exact requirement and even provides a **Quick Start Prompt**.

Lab_4_VibeCoding_Student_handout

Task 1 goal

Currently:

```python
self.enemy = Enemy(ROWS-1, COLS-1)
```

We need to change this to:

- **3 enemies total**
- Start from **different corners**
- Store them in:

```python
self.enemies
```

- Each enemy independently runs `bfs()` during `update()`.
- Collision must work with **all enemies**.
- All enemies must be drawn.

Important: how we'll do the assignment

Since this is specifically a **Vibe Coding assignment**, don't manually edit the code yet.

Use this prompt in ChatGPT/Copilot/Cursor/whatever Vibe Coding tool you're using:

Task 1 Vibe Coding Prompt

In my maze-chase pygame game, I have one Enemy that uses BFS to chase the player. Add two more Enemy instances starting at different maze corners. Each should independently call bfs() on every update. Store all enemies in a self.enemies list and loop over them for updating, drawing, and collision detection.

Do not implement any other task from the README. Keep the existing game behaviour unchanged otherwise.

After the AI changes the code

Test these specifically:

- [ ] Game starts normally.
- [ ] There are **3 enemies**.
- [ ] They start at different corners.
- [ ] All 3 move toward the player.
- [ ] They can navigate the maze.
- [ ] Player collision with **any** enemy causes `CAUGHT!`.
- [ ] `R` still restarts the game.
- [ ] No errors appear in the terminal.

Then commit it

Bash

```
git add .
git commit -m "Task 1 - Add multiple enemies"
```

Don't start Task 2 yet. We want one clean commit per task, exactly as the assignment requires.

Lab_4_VibeCoding_Student_handout

Also, keep the **AI chat history** from this interaction. The README explicitly asks for the LLM review/chat link, and the lab requires the exported chat history. Lab_4_VibeCoding_Student_handout

ok it works

Good. **Task 1 is complete.** One small victory for software engineering.

Now commit Task 1

Run:

```bash
git status
```

Then:

```bash
git add .
git commit -m "Task 1 - Add multiple enemies"
```

Then verify:

```bash
git log --oneline -3
```

You should see your new commit at the top.

Then Task 2: Speed Up Over Time

Requirement:

- Every **15 seconds**, enemies become faster.
- Reduce `enemy.move_interval` by **2**.
- Minimum value is **5**.
- Use `pygame.time.get_ticks()` .
- Show the **current speed tier in the HUD**.

Use this as the next Vibe Coding prompt:

Task 2 Vibe Coding Prompt

Implement only the "Speed Up Over Time" feature from the README.

Every 15 seconds of gameplay, decrease the move_interval of all enemies by 2, with a minimum move_interval of 5.

Use pygame.time.get_ticks() to track elapsed time rather than adding another frame-based timer.

Add a speed tier indicator to the HUD showing the current difficulty/speed tier.

Do not implement the power pellet or survival score yet. Preserve all existing functionality, including the three-enemy system and BFS behaviour.

Test Task 2

Check:

- [ ] Game starts with 3 enemies.
- [ ] After 15 seconds, enemies visibly move faster.
- [ ] After another 15 seconds, they speed up again.
- [ ] Speed never goes below `5`.
- [ ] HUD displays the speed tier.
- [ ] No existing Task 1 behaviour breaks.

Once it works:

```bash
git add .
git commit -m "Task 2 - Add difficulty ramp"
```

Then we'll move to **Task 3: Power Pellet**.

Today 6:42 PM

it works , next

Task 3: Power Pellet

Now we add **only the Power Pellet feature**.

Requirement

- Add **one yellow power pellet** somewhere in the maze.
- When the player collects it:
  - Enemies become frozen.
  - Freeze lasts **5 seconds / 300 frames**.
  - Frozen enemies **do not move**.
  - Show a **frozen indicator** on the enemy.

Because we already have 3 enemies from Task 1, the safest interpretation is: **the power pellet freezes all active enemies**. Otherwise the original singular wording becomes awkward after Task 1.

Vibe Coding prompt

Task 3 Vibe Coding Prompt

Implement only the "Power Pellet" feature from the README.

Add a yellow power pellet at a suitable walkable location in the maze. When the player collides with and collects the pellet, freeze all current enemies for 5 seconds (300 frames).

While frozen, enemies must not move or update their BFS movement. Display a clear frozen indicator on each frozen enemy.

The pellet should be collectible only once.

Preserve all existing functionality from Tasks 1 and 2, including the three enemies, BFS chasing, speed-up-over-time system, collision detection, restart behaviour, and speed-tier HUD.

Do not implement the Survival Score feature yet.

Test it carefully

- [ ] Yellow pellet appears.
- [ ] Player can reach it.
- [ ] Pellet disappears after collection.
- [ ] **All 3 enemies stop moving.**
- [ ] Frozen indicator appears.
- [ ] They remain frozen for approximately 5 seconds.
- [ ] They resume chasing afterwards.
- [ ] Speed-tier system still works.
- [ ] Restart resets the pellet/frozen state.
- [ ] No existing functionality breaks.

Then commit

```bash
git add .
git commit -m "Task 3 - Add power pellet"
```

After that, there's only **Task 4: Survival Score**, followed by final testing, the AFTER recording, and pushing the repository.

it works , next

Task 4: Survival Score

Last feature. Humanity survives another timer.

Requirement

- While the player is alive:
  - Increase `self.score` by **1 every frame**.
- HUD displays:

```
Plain text

Survived: Xs
```

- Calculate:

```
Python

X = self.score // 60
```

- Show the final survival score on the **game-over overlay**.
- Preserve Tasks 1–3.

Vibe Coding prompt

Task 4 Vibe Coding Prompt

Implement only the "Survival Score" feature from the README.

While the player is alive, increment self.score by 1 every frame.

Display the survival time in the HUD using exactly:
Survived: Xs

where X = self.score // 60.

When the game ends, display the final survival score/time on the game-over overlay.

Preserve all existing functionality from Tasks 1, 2, and 3, including the three enemies, BFS chasing, speed-up-over-time system, speed-tier HUD, power pellet, frozen-enemy behaviour, collision detection, and restart behaviour.

Do not make unrelated changes.

Test it

- [ ] `Survived: 0s` appears initially.
- [ ] Score increases while the player is alive.
- [ ] It changes approximately once per second.

- [ ] Score stops increasing after game over.
- [ ] Final survival time appears on the game-over screen.
- [ ] Restart resets the score.
- [ ] Tasks 1–3 still work.

Then commit

```bash
git add .
git commit -m "Task 4 - Add survival score"
```

After this commit, **do not change the code further** until we do the final verification.

works, next

Final phase: verify, record, push

All 4 development tasks are now done.

1. Verify Git history

Run:

```bash
git log --oneline --all
```

You should have separate commits roughly like:

```
Task 4 - Add survival score
Task 3 - Add power pellet
Task 2 - Add difficulty ramp
Task 1 - Add multiple enemies
```

If you also made a baseline commit, that is fine.

2. Do one final full test

Start the game and verify:

- [ ] 3 enemies work
- [ ] BFS chasing works
- [ ] Enemies speed up every 15 seconds

- [ ] Speed tier appears
- [ ] Power pellet works
- [ ] Enemies freeze for 5 seconds
- [ ] Frozen indicator appears
- [ ] Survival score increases
- [ ] Final score appears after game over
- [ ] `R` correctly restarts everything
- [ ] No Python errors

3. Record the AFTER video

The handout requires another **10-second recording after all changes**. Lab_4_VibeCoding_Student_handout

Your 20-second recording is also fine as evidence. For the AFTER recording, ideally show several implemented features in one continuous clip:

3 enemies → pellet/frozen state → HUD showing score/speed

You don't need to demonstrate every feature separately in 10 seconds.

4. Push to your personal repository

First:

```bash
git status
```

Then:

```bash
git push
```

If this is your first push and Git asks for the upstream branch:

```bash
git push -u origin main
```

Do not create a PR to `SETAPESU26`, as explicitly instructed by the lab. Lab_4_VibeCoding_Student_handout

5. Prepare the Lab-4 folder

Your submission should contain:

Plain text

```
Lab-4/
│
├── before.mp4
├── after.mp4
├── maze-chase/
│   ├── main.py
│   ├── ...
│
└── chat_history.pdf
```

The required deliverables are **before/after videos, updated code, and exported chat history**.

Lab_4_VibeCoding_Student_handout

One thing left

Don't export the chat history yet. First do the Git push and final test. Then we'll prepare the **chat-history document and submission structure** properly, including the prompts for all four tasks.